HW09

Sam Ly

April 10, 2026

Chapter 18

https://github.com/samlyme/Assignments/tree/cs4080/ch18/main

1. We could reduce our binary operators even further than we did here. Which other instructions can you eliminate, and how would the compiler cope with their absence?

   The two operations that could potentially be removed are the OP NIL, TRUE, and FALSE, and the either OP GREATER or OP LESS.

   We can remove all the data creation ops simply because we already have the OP CONSTANT. Then, for OP GREATOR, the compiler can just reorder the elements and use an OP LESS and vice versa.

2. Conversely, we can improve the speed of our bytecode VM by adding more specific instructions that correspond to higher-level operations. What instructions would you define to speed up the kind of user code we added support for in this chapter?

   We can add more boolean operations like NOT EQUAL and GREATER THAN OR EQUAL.

Chapter 19

1. Each string requires two separate dynamic allocations—one for the ObjString and a second for the character array. Accessing the characters from a value requires two pointer indirections, which can be bad for performance. A more efficient solution relies on a technique called flexible array members. Use that to store the ObjString and its character array in a single contiguous allocation.

```
static ObjString* allocateString(char* chars, int length) {
  ObjString* string = (ObjString*)allocateObject(sizeof(ObjString) + (length+1)*sizeof(char), OBJ_S
  string->length = length;
  for (int i = 0; i < length; i++) {
    string->chars[i] = chars[i];
  }
  string->chars[length] = '\0';
  return string;
}
```

2. When we create the ObjString for each string literal, we copy the characters onto the heap. That way, when the string is later freed, we know it is safe to free the characters too.

   This is done by creating a new “string literal” function, along with a new field inside of OjbString that tracks if it is a string literal. Thus, when freeing the objects, we can check if the object is a literal, and only free the actual Object, but if it is not, then we also need to free the ‘char*’.

```
// object.c
ObjString* stringLiteral(const char* chars, int length) {
  ObjString* literal = ALLOCATE_OBJ(ObjString, OBJ_STRING);
  literal->isLiteral = true;
  literal->length = length;
  literal->chars = chars;
  return literal;
}

// object.h
struct ObjString {
    Obj obj;
    int length;
    bool isLiteral;
    char* chars;
};

ObjString* takeString(char* chars, int length);
ObjString* copyString(const char* chars, int length);
ObjString* stringLiteral(const char* chars, int length);

// compiler.c
static void string() {
    emitConstant(OBJ_VAL(stringLiteral(parser.previous.start + 1,
                                       parser.previous.length - 2)));
}

// memory.c
void freeObject(Obj* object) {
    switch (object->type) {
        case OBJ_STRING: {
            ObjString* string = (ObjString*)object;
            if (!string->isLiteral) {
                FREE_ARRAY(char, string->chars, string->length+1);
            }

            FREE(ObjString, object);
            break;
        }
    }
}
```

3. If Lox was your language, what would you have it do when a user tries to use + with one string operand and the other some other type? Justify your choice. What do other languages do?

   I personally would allow the '+' operator on a string and some other type, then implicitly cast the other type to a string. Most other high-level dynamically typed languages like JS and Python also allow for this behavior. This makes sense because we have already established that types are a bit loose in this language, so the extra freedom of being able to implicitly concat strings and non-string types would be an expected feature of the language.